Lecture Slip, CS 135
March 18, 2026, Wednesday

Name:	
EmpID:	

1. Define a function `append`, given an array of integers with capacity, and number of elements size, and an element, if the element is not in the array yet, add it to the end of array, size is increased by 1. The return type of the above function is void.

2. In main function, define an array of integers with capacity 10. Call `append` to initialize the array with elements of even integers in [2, 10].

   Call `append` function to try to add 2 to the above array. What will happen?

   Call `append` to put even integers in [12, 22] to array.

   Print out all elements of array.

Answer:

1 Input Parameters of Array Function Must Include Array Name and Array Size

The input parameters of a function to work with an array must include at least the **array name** and the **array size**.

Reason:

1. The array name represents its starting address, that is, the address of the first element.

   With an address, we can access the element residing in it.

2. To access element at index i, calculate the ith element's address by

   initial address + i * memory size of array type

3. Need array size to know the number of elements in this array, or we cannot find out the value of the last index, which is size -1, where the first index is 0.

4. As an analog, suppose a group of people living in consecutive rooms. Each member occupies a single room. Once we know the first member's room number (aka address) and the number of members (aka size) of this group, we can access each member of this group.

Depending on the functionality, you may need to add more parameters. For example, the `exist` function checks whether an element is in the array or not. For such a function, besides the array name and the array size, there should be a parameter for the element to be searched.

2 Declare and Define `print` Function

2.1 Declare `print` Function

Function `print` displays all elements of an array.

```
void print(int arr[], int size);
//int arr[] declares an integer array
//int size declares the array size
```

2.2 Define `print` Function

Wrong implementation: `print` function does not have a parameter for array size. Then try to calculate array size inside `print` function.

Reason: when an array is passed as an argument to a function, it is **not** passed by value where each element is copied. Instead, the array undergoes an implicit conversion – known as **decay** – into a pointer to its first element. Consequently, `sizeof(arr)` within function `print` does **not** return the memory size of the original array; it returns the size of a pointer of the array type. In this example, `sizeof(arr)` returns the memory size of an integer pointer `int*`. In a 32-bit system, `sizeof(int*)` returns 4 bytes (32 bits), and 8 bytes in a 64-bit system (64 bits).

```
void print(int arr[]) {
    int size = sizeof(arr) / sizeof(arr[0]);
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
     }

    std::cout << std::endl;
}
```

WRONG

Here is a fix.

```
void print(int arr[], int size) {
  for (int i = 0; i < size; i++) {
    std::cout << arr[i] << " ";
  }

  std::cout << std::endl;
}
```

CORRECT

3 Declare and Define `exist` Function

3.1 Declare `exist` Function

The `exist` function checks whether a specific element is present in an array.

To perform this check, the function requires:

- The array itself.
- The array size.
- The element to search for.

Since this function does not add or delete elements from the array, the array size is not changed, pass-by-reference is unnecessary for the size parameter.

The return type is `bool`: it returns true if the element is found, and false otherwise.

The declaration of `exist` function is as follows.

```
bool exist(int arr[], int size, int element);
```

3.2 Define `exist` Function

- Once the element to search is found, return true immediately.
- Only **after** searching **all** elements of the array and do **not** find **any** match, can we return false.

  We **cannot** return false immediately when a **mismatch** is found.

The following implementation is incorrect due to two main errors:

Logical error: the following version returns false immediately upon finding the first mismatch. This logic is flawed because a value can only be confirmed as absent after all elements in the array have been checked. A correct implementation should only return false if the entire array has been traversed without finding a match.

Furthermore, with the following implementation, only the first element arr[0] of the array is checked. If it is a match, the function immediately returns true; otherwise, it returns false. All other elements in the array remain unchecked.

Compilation error: both return statements are located inside for-statement. When the entry condition `i < size` in the for-statement header is not satisfied – for example, size is 0 for an array without any element filled – no code in for-statement body is run. Consequently, the function reaches its end without returning any value (true or false) after the loop terminates. This violates the rule for functions with a non-void return type, which must guarantee a return value in all possible execution paths.

```
bool exist(int arr[], int size, int element) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == element) {
           return true;
        }
        else return false; //WRONG, 1000% WRONG
    }
}
```

WRONG

Here is a fix of the above code.

```
bool exist(int arr[], int size, int element) {
  for (int i = 0; i < size; i++) {
    if (arr[i] == element) {
      return true;
    }
  }

  return false;
}
```

CORRECT

4 Declare and Define `append` Function

4.1 Declare `append` Function

The `append` function requires the following input parameters:

1. An integer array

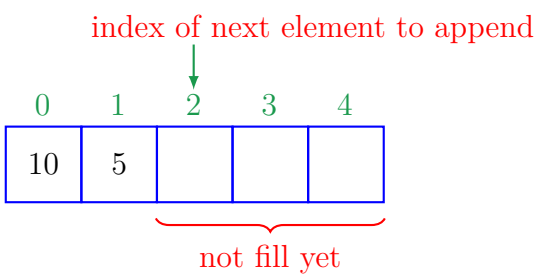
   The array name also represents its starting address, that is, the address of the first element of the array.

2. The array size

   (a) The array size specifies the current number of elements in the array. A size of zero means the array is empty.

   (b) Because array indices start at 0, the size value also indicates the index where the new element will be inserted.

   For example, if an array has a capacity of 5 and currently contains 10 and 5, the index of the element to be appended is 2.



   (c) In the `append` function, the return type is void by setting. After inserting an element, the array size is changed and the updated value needs to be carried back to its caller. To achieve this, the **array size parameter** must be **passed by reference**, as in `int& size`, in the function definition's header.

3. The array capacity

   (a) This defines the maximum number of elements the array can hold. Before appending a new element, we must check whether the capacity has been reached. If the size – the current number of elements in the array – equals the capacity, no additional elements can be added.

(b) To add elements to an array, `capacity` needs to be a parameter so that the maximum number of elements for an array will not be exceeded.

4. The element to append

5. The function declaration is as follows.

```
void append(int arr[], int capacity, int& size, int element);
```

4.2 Define `append` Function

1. If the array is full (i.e., the current size equals the capacity), return to the caller immediately. Since the `append` function's return type is `void`, a simple `return;` statement is sufficient.

2. If the element is already present in the array, no insertion is needed per the problem's requests, so return immediately.

3. If we have not returned in the previous steps, it confirms that the array is not full and the element is not already present. In this case:

   (a) Add the new element to the end of the array.

   (b) Increment the size by 1 to reflect the addition of a new element. The `size` variable now indicates the index of the next available position at the end of the array if the array is not full.

```
void append(int arr[], int capacity, int& size, int element) {
    if (size == capacity) {
       return;
    }

    if (exist(arr, size, element)) {
       return;
    }

    //the index of the next available location in the array is size
    arr[size] = element;
    size++;
}
```

5 Define `main` Function

1. Declare an integer array with capacity 10.

   The array capacity needs to be const integer variable. By convention, a const variable's name is written in all capital letters.

```
const int CAPACITY = 10; //Warning:  keyword const cannot be omitted
int arr[CAPACITY];
```

2. Initially, the array contains no elements. Therefore, `size` (the number of elements currently in the array) is initialized to zero.

```
int size = 0;
```

3. Use a loop to call the append function and add all even numbers in the range [2, 10] to the array.

   (a) The first even number in [2, 10] is 2. Hence, initialize loop variable `value` to be 2.

   (b) As long as `value` is less than or equal to 10 – the upper bound of the range – call `append` function to add `value` to the array.

   (c) Then increment loop variable `value` by 2 to move to the next even number.

   (d) Note that the size parameter is passed by reference in function `append`, meaning any changes to the array size within function `append` are preserved when control returns to its caller, in this example, `main` function. After this process, the size of the array will be 5.

```
for (int value = 2; value < 10; value += 2) {
    append(arr, CAPACITY, size, value);
}
```

4. Print out the current elements of the array by calling `print` function.

```
print(arr, size);
```

5. Append element 2 to the array.

```
append(arr, CAPACITY, size, 2);
```

   By assumption, the array does not accept redundant elements. So, in `append` function, `exist` function is called to test whether an element is in the array or not; if yes, no insertion.

   Element 2 is inserted when appending even integers in [2, 10] to the array as in 3d. As a result, the elements of `arr` are 2, 4, 6, 8, 10.

   Call `append` function on 2 will not insert a redundant copy of 2 to the array. The elements of the array are still 2, 4, 6, 8, 10.

6. Call `print` function to print elements in the array to verify that 2 is not inserted more than once.

```
print(arr, size);
```

7. Use a loop to call the append function and add all even numbers in the range [12, 22] to the array.

   (a) The first even number in [12, 22] is 12. Hence, initialize loop variable `value` to be 12.

   (b) As long as `value` is less than or equal to 22 – the upper bound of the range – call `append` function to add `value` to the array.

   (c) Then increment loop variable `value` by 2 to move to the next even number.

(d) After calling `append` to add even integers in [2, 10] as shown in 3d, the array has 5 elements already.

The array capacity is 10, so only five more elements can be inserted. As a result, element 22 is not appended to the array due to capacity limit.

Note that `size` parameter is passed by reference in `append` function, so the change on the array `size` is carried back to the caller `main` function. After the following statement, size is changed to 10.

```
for (int value = 12; value <= 22; value += 2) {
    append(arr, CAPACITY, size, value);
}
```

8. Call `print` function to print out the elements in the array.

```
print(arr, size);
```

6 Complete Code

Onlinegdb link is `https://onlinegdb.com/OgUHK16Y-`.

```
//code link: https://onlinegdb.com/OgUHK16Y-
//purpose: define append function to do the following:
//(1) Given an integer array with its capacity and size,
//    together with an element to insert.
//(2) If the array is not full and
//    the element to insert is not in the array,
//    append the element to the end of the array.
//(3) The return of function append is set to be void.

//In main function,
//(1) Declare an integer array with capacity 10.
//(2) Call append function to add all even integers in [2, 10]
//    to the array.
//(3) Print the elements in the array.
//(4) Call append function to try to add 2 again to the array.
//(5) Print the elements in the array.
//(6) Call append function to try to add all even integers
//    in [12, 22] to the array.
//(7) Print the elements in the array.

#include <iostream>
#include <string>

//sample input/output:
//contents of arr after add even integers in [2, 10]
```

```
//2 4 6 8 10
//contents of arr after trying to add 2 when no redundant element is allowed
//2 4 6 8 10
//contents of arr after trying to add even integers in [12, 22]
//2 4 6 8 10 12 14 16 18 20

bool exist(int arr[], int size, int elm);

//append to the end of array if the element is not in the array and the array has sufficient capacity
void append(int arr[], int capacity, int& size, int elm);

void print(int arr[], int size);

int main() {
    //In main function, define an array of integers with capacity 10.
    const int CAPACITY = 10;
    int arr[CAPACITY];

    int size = 0; //current number of element of the array

    //Call append to initialize the array
    //with elements of even integers in [2, 10].
    for (int value = 2; value <= 10; value += 2) {
        append(arr, CAPACITY, size, value);
    }

    std::cout << "contents of arr after add even integers in [2, 10]" << std::endl;
    print(arr, size);

     //Call append function to try to add 2 to the above array. What will happen?
    append(arr, CAPACITY, size, 2);

    std::cout << "contents of arr after trying to add 2 when no redundant element is allowed" << std::endl;
    print(arr, size);

    //Call append to put even integers in [12, 22] to array.
    for (int value = 12; value <= 22; value += 2) {
        append(arr, CAPACITY, size, value);
    }

    std::cout << "contents of arr after trying to add even integers in [12, 22]" << std::endl;
    //Print out all elements of array.
    print(arr, size);
```

```

    return 0;
}

//if elm exists in arr, return true, otherwise, return false
bool exist(int arr[], int size, int elm) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == elm) {
           return true;
        }
        //else return false; //WRONG, 1000% WRONG
    }

    return false;
}

//append elm to the end of arr if
//(1) arr does not reach its capacity
//(2) elm is not in arr, that is, no other element in arr equals elm
void append(int arr[], int capacity, int& size, int elm) {
     if (size == capacity) {
        return;
     }

     if (exist(arr, size, elm)) {
        return;
     }

     arr[size] = elm;
     size++;
}

//print elements in an array
void print(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;
}
```

7 If the element is not in the array, how do I add it later?

When adding elements to an array, we must distinguish between two concepts:

- CAPACITY: The maximum number of elements an array can hold.

  In C-style arrays, capacity must be a constant. By convention, constant names use uppercase letters separated by underscores (e.g., `MAX_SIZE` or CAPACITY). For example, C++ uses `M_PI` in the `cmath` library to represent π.

  Crucially, the size of an array cannot be larger than its capacity.

- `size`: The number of elements currently stored in the array.

  Elements must be added starting from index zero and placed in consecutive cells.

The Hotel Analogy
Imagine a hotel with 10 rooms; its capacity is 10.

- The first customer stays in room 0.
- The second customer stays in room 1.
- The third customer stays in room 2.

There are now three customers in the hotel, so the size is 3.
Before any elements are added, the size is 0.
Note that the `size` conveniently matches the index where the next element should be placed.